Edge — a Django view — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

- Hint 1 — a view is a function too
- Hint 2 — running it without a full project

Hint 1 — a view is a function too

A Django **view** is a function that takes a `request` and returns a response — the same idea as Flask, with Django's objects:

```
from django.http import JsonResponse
import json
from pipeline import classify

def classify_view(request):
    body = json.loads(request.body)
    if not body.get("text","").strip():
        return JsonResponse({"error":"empty text"}, status=400)
    return JsonResponse({"label": classify(body["text"])})
```

A `urls.py` maps a path to it: `urlpatterns = [path("classify", classify_view)]`. In a real project you'd `django-admin startproject` and drop these in — you don't hand-build the layout.

Hint 2 — running it without a full project

To test the view without `manage.py runserver`, bootstrap Django inline in one file:

```
from django.conf import settings
settings.configure(DEBUG=True, ROOT_URLCONF=__name__, ALLOWED_HOSTS=["*"],
                   DATABASES={}, INSTALLED_APPS=[])
import django; django.setup()
# ... define classify_view and urlpatterns ...
from django.test import Client
Client().post("/classify", data='{"text":"amazing"}',
              content_type="application/json").json()   # {"label":"positive"}
```

Notice how much more setup this took than Flask or FastAPI — that's the signal that Django is for *products* (where the auth, admin, and ORM pay for the weight), not single endpoints. The full code is in the Solution.